

Q1/ Recursive Algorithms

Given a sentence (treated as a list of words), a starting index s , and an ending index e .

Goal: return only the words between s and e .

Steps:

1/ If $s > e$ then,
no words to take, so the algorithm returns an empty sentence.
return empty;

2/ If $s = e$ then,
exactly one word in the range,
range = word + 1;

3/ (general recursive case): Otherwise
word = ^{index}[s];
(take the word at position s ;

- recursively obtain the part of the sentence from index $s+1$ to e .
- combine the word taken at step 1 with the sentence returned by the recursive call.
- ~~Result~~ Return this combined result.

* Recurrence Relation:

Let $n = e - s + 1$, (no. of words needed to extract).

- At each recursive call:

- Take one word = constant time

- Combine with Result = constant time

- ^{Make} One recursive call on remaining $n-1$ words

So, recurrence is:

$$T(n) = T(n-1) + c$$

Base case:

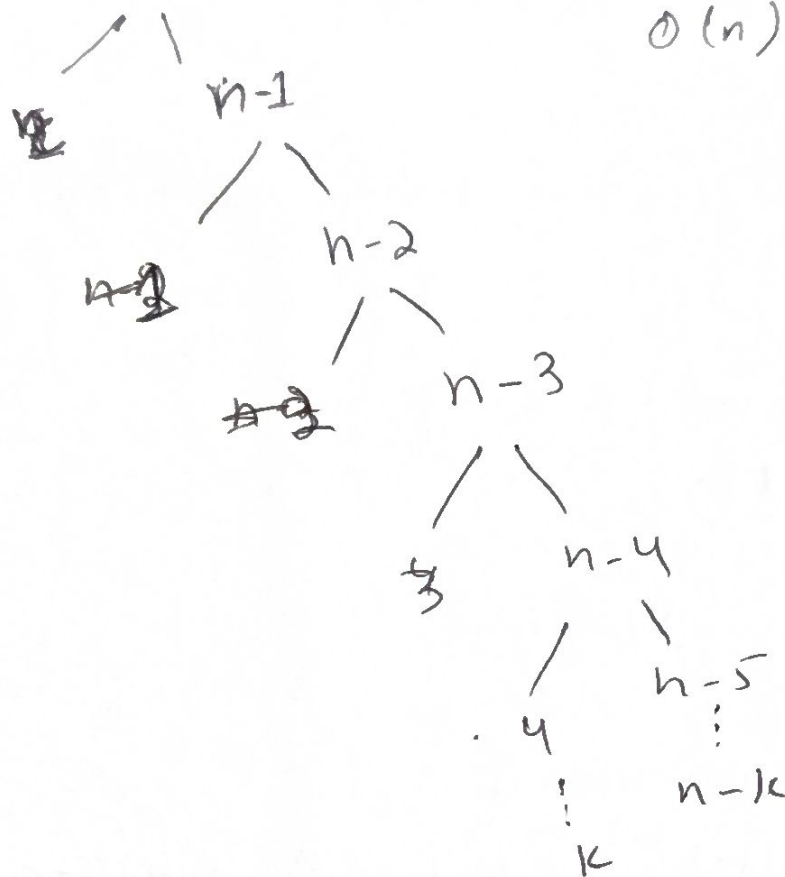
$$T(1) = d$$

'c' and 'd' are constants.

* Solving recurrence

$$T(n) = T(n-1) + c$$

$$O(n)$$



n -levels
↓
each contributes c
work = linear in n .

~~$n(n+1)$~~

$$n \frac{(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

$n^2 \rightarrow$ complexity

Recurrence Repeatedly:-

$$T(n) = T(n-1) + c$$

$$T(n) = T(n-2) + 2c$$

$$T(n) = T(n-3) + 3c$$

⋮

$$T(n) = T(1) + (n-1)c$$

$$T(n) = O(n^2) \rightarrow \text{Time Complexity.}$$

Q2/ Sorted Array 'A' of size n has been clockwise shifted (rotated) by k positions.

→ Find smallest element / find the rotation point, determine k .
 Approach → modified binary search.

* Algo (Pseudocode)

Algorithm FindRotationPoint (A, left, right):

If left == right:
 return A[left]

mid = (left + right) / 2

If A[mid] > A[right]:

return FindRotationPoint (A, mid + 1, right)

Else:

return FindRotationPoint (A, left, mid - 1)

If max > right element.
 // pivot is in the right half
 then left must be in the left half including mid.
 // pivot is in the left half (including mid)
 # otherwise max is in the left part and it is before mid.

* Recurrence Relation

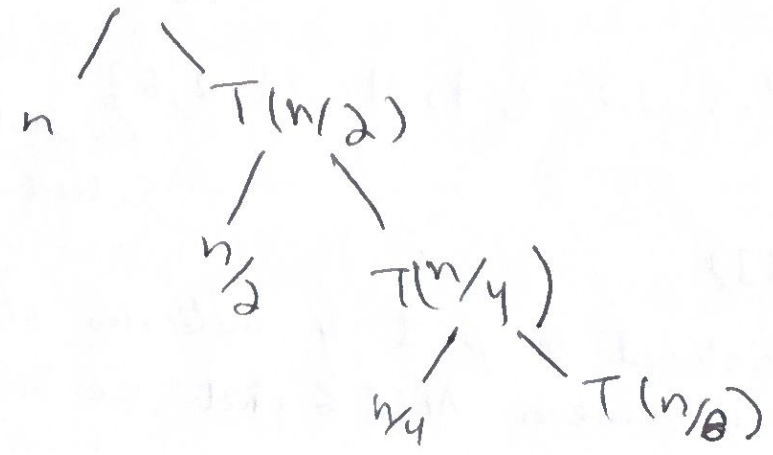
$$T(n) = T(n/2) + c$$

$$T(1) = d$$

c, d = constants

a Recursive Tree Method

$$T(n) = T(n/2) + n$$



→ Original sorted:-

5, 15, 27, 29, 35, 42

→ After shifting $k=2$:

35, 42, 5, 15, 27, 29

Max = 42

→ After shifting $k=4$:

27, 29, 35, 42, 5, 15

Max = 42

$$1 \rightarrow \frac{1}{2} \rightarrow \frac{1}{4} \rightarrow \dots \rightarrow 1$$

$$\log_2(n)$$

$$T(n) = n \log n + d = O(\log n)$$

Recurrence -

$$T(n) = T\left(\frac{n}{2}\right) + c$$

$$T(n) = T\left(\frac{n}{4}\right) + 2c$$

$$T(n) = T\left(\frac{n}{8}\right) + 3c$$

After k expansions -

$$T(n) = T\left(\frac{n}{2^k}\right) + kc$$

$$\frac{n}{2^k} = 1, \quad k = \log n.$$

Thus,

$$T(n) = T(1) + c \log n$$

Complexity $\rightarrow T(n) = \Theta(\log n)$

- 3/ - Partition (illustrate)
- Quick sort runs $O(n^2)$ - (worst case)

$$A = \{13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6\}$$

Initial $\rightarrow$

$\hookrightarrow$ pivot
(last element).

$$\text{pivot} = 6 (A[11])$$

$i=0$ (i initialized to $p-1$ if indicates start at 1)

run $j=1 \dots 10$: whenever $A[j] \leq \text{pivot}$ we increment i and swap $A[i]$ & $A[j]$.

3) $j=2: A[2] = 22 > 6 \rightarrow$ no action (i=0)
array: [5, 19, 9, 13, 12, 8, 7, 4, 21, 2, 6]

4) $j=3: A[3] = 19 > 6 \rightarrow$ no action (i=1)
array unchanged.

5) $j=4: A[4] = 12 > 6 \rightarrow$ no action (i=1)

6) $j=5: A[5] = 8 > 6 \rightarrow i=2; \text{ swap } A[2] \text{ and } A[5].$

After swap: [5, 19, 8, 13, 12, 2, 7, 4, 21, 2, 6]

7) $j=6: A[6] = 12 > 6 \rightarrow$ no action (i=1)

8) $j=7: A[7] = 2 > 6 \rightarrow$ no action (i=1)

9) $j=8: A[8] = 7 > 6 \rightarrow$ no action (i=1)

10) $j=9: A[9] = 4 \leq 6 \rightarrow i=2; \text{ swap } A[2] \text{ and } A[9].$

After swap: [5, 4, 8, 13, 12, 2, 7, 19, 21, 2, 6]

11) $j=10: A[10] = 21 > 6 \rightarrow$ no action (i=2)

12) $j=11: A[11] = 2 \leq 6 \rightarrow i=3; \text{ swap } A[3] \text{ and } A[11].$

After swap: [5, 4, 2, 13, 12, 8, 7, 19, 21, 9, 6]

End of loop. Now place pivot to $i+1$ position.

Swap $A[i+1] = A[4]$ (13) with pivot $A[11]$ (6):

* Array after partition:

$A = [5, 4, 2, 6, 13, 8, 7, 19, 21, 9, 13]$

* Quick sort worst case $O(n^2)$ - distinct element sorted in desc order.

* if array (distinct element) sorted desc order

pivot = first & last element

then complexity = n^2

* On each partition:-

→ desc array: $A = [n, n-1, \dots, 2, 1]$.

→ with pivot = $A[n] = 1$ (smallest element).

while partition every $A[j]$ ($j = 1 \dots n-1$) is $>$ pivot, none satisfy $A[j] \leq$ pivot.

Thus, i stays at 0, and at the end we swap pivot with $A[1]$.

The pivot (smallest element) ends up at position 1. The split is: left

subarray size 0, right subarray size $n-1$.

→ That is, PARTITION produces an extremely unbalanced split: sizes 0 and $n-1$.

* Recurrence for running time:

$T(n) \rightarrow$ time to quicksort / distinct element in worst-case ordering.

$O(n)$ time (partition - subarray once). QS recurses on larger side of size $n-1$ and the small side of size 0. So recurse.

$$T(n) = T(n-1) + O(n), \quad T(1) = O(1).$$

Then,

$$T(n) = \{T(n-1) + cn = (T(n-2) + c(n-1)) + cn = T(n-2) + c(n-1) + cn = \dots$$

$$\dots = T(1) + c \sum_{k=2}^n k = O(1) + c \cdot O(n(n+1)/2)$$

$$\text{Simply, } T(n) = c \cdot n + c \cdot (n-1) + \dots + c \cdot 1 = O(n^2).$$

* ~~Recursion~~

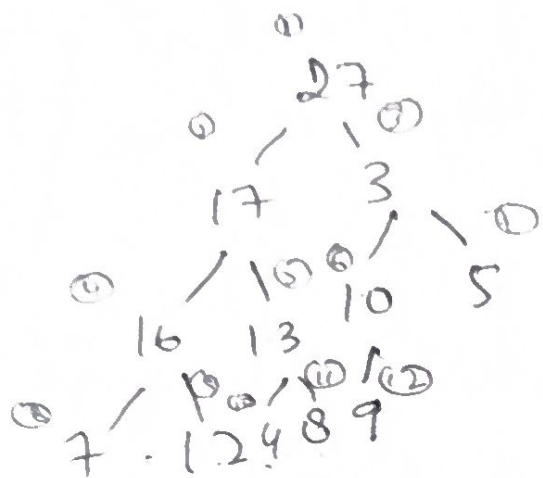
Worst $O(n^2) \rightarrow$ bcz of pivot choice Worst case element

Final array:- $[5, 4, 2, 6, 12, 8, 7, 19, 21, 9, 13]$
pivot = 6 (index: 4).

Q.4/ (Heapify) Heap-sort

1/ Max-heapify (A, 3)

A = { 27, 17, 3, 16, 13, 10, 5, 7, 12, 4, 8, 9 }



Max Heapify (A, 3):

- Node at index 3=3

children
left [6] = 10

right i=[7]=5

→ After i=1 → 27 child compare to its children rotate & swap.

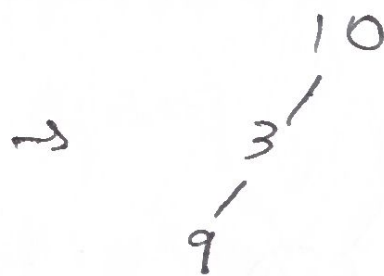
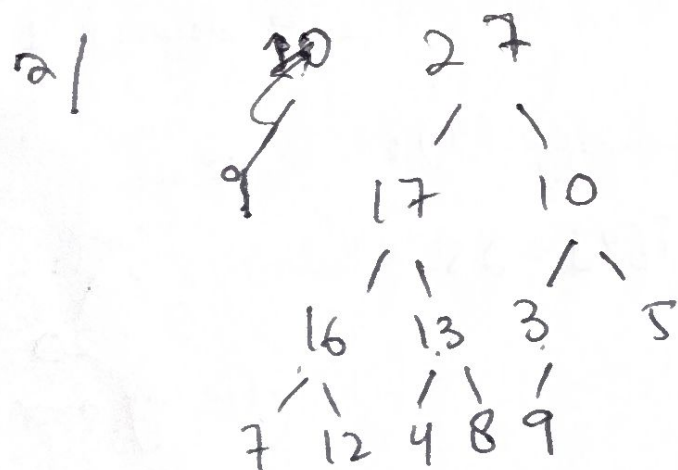
largest = 10 at i=6

Swap A[3] ↔ A[6] (indexing)

New array (after swap)

{ 27, 17, 10, 16, 13, 3, 5, 7, 12, 4, 8, 9 }

→ Max-heapify at index = 6



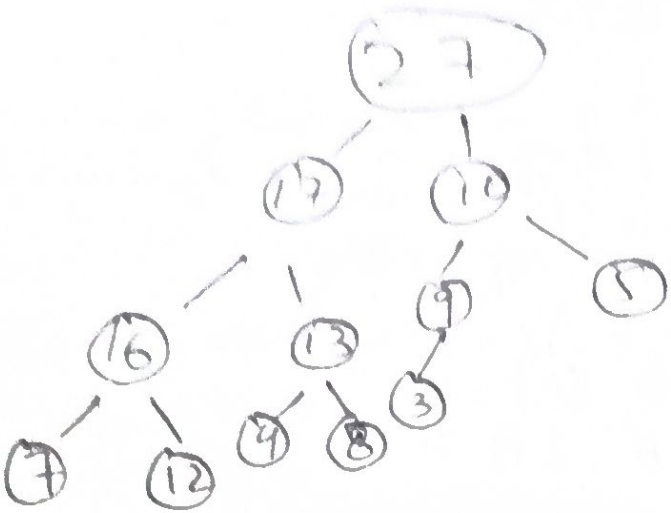
Swap A[6] ↔ A[12]

Array:- 27, 17, 10, 16, 13, 9, 5, 7, 12, 4, 8, 3

∴ at index 6 is now 9 → its children (outside array) / heapify stop.

Max Heapify (A, 3)

{ 27, 17, 10, 16, 13, 9, 6, 7, 12, 4, 8, 3 }



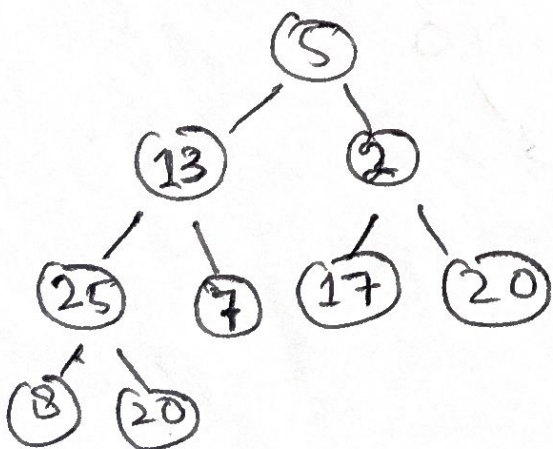
② Heap-Sort on $A = \{ 5, 13, 2, 25, 7, 17, 20, 8, 20 \}$

1/ Build Max-Heap

2/ Swap root (repeatedly) with last element and heapify reduced heap.

① Max-Heap

Initial tree-



Start heapifying from $\frac{n}{2} = \frac{9}{2} \rightarrow 4$
 $\rightarrow 4$ down to 1.

Heapify (4):

$i[4] = 25$

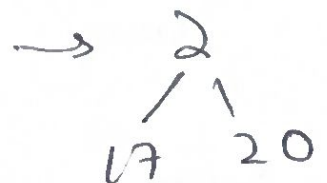
Tree-



already max heap
no swapping required.

Heapify (3):

$i[3] = 2$



Largest = 20 $\rightarrow$ swap with $A[7]$

Array: { 5, 13, 20, 25, 7, 17, 2, 8, 20 } $\begin{matrix} 20 \\ 17 \end{matrix}$

Heapify process: $7 \rightarrow 17 \rightarrow 17$

• Heapify(2):

$i(2) = 13$



Input $\rightarrow$ swap $A(2) \rightarrow A(1)$

new tree



Array: $\{5, 25, 20, 20, 7, 17, 2, 8, 13\}$

Heapify(9) $\rightarrow$ leaf.

• Heapify(1):

$i = 1$ [5] ^{element}

Tree



Input = 25 swap $\rightarrow$



Array now $\{25, 5, 20, 20, 7, 17, 2, 8, 13\}$

Heapify(2): children 20 & 7 $\rightarrow$ largest = 20 $\rightarrow$ swap

Array $\{25, 20, 20, 5, 7, 17, 2, 8, 13\}$

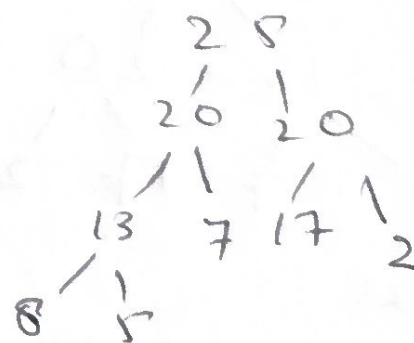
H(4): children 8 & 13 $\rightarrow$ largest = 13 $\rightarrow$ swap

Array $\{25, 20, 20, 13, 7, 17, 2, 8, 5\}$

Heapify steps-

Max heap with-

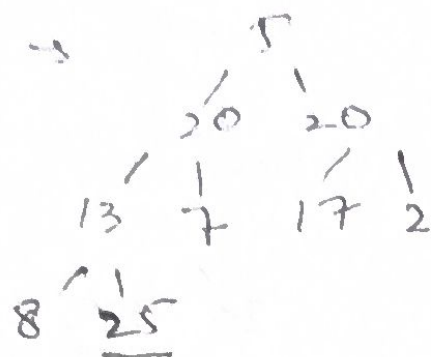
$\{25, 20, 20, 13, 7, 17, 2, 8, 5\}$



② Heap Sort

① —!

Swap $A[1] \leftrightarrow A[9]$

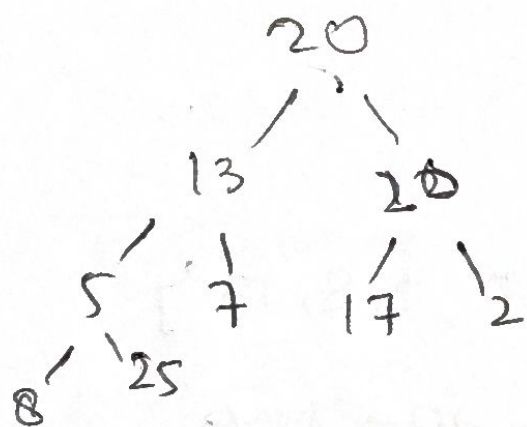


Heapify (1) override 8 r

Largest child = 20 $\rightarrow$ swap

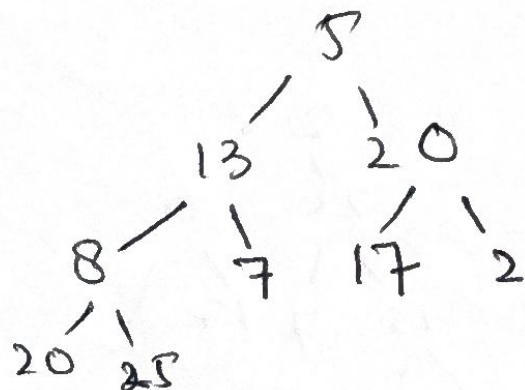
$\rightarrow \{20, 5, 20, 13, 7, 17, 2, 8, 25\}$

Heapify (2): Largest child = 13 $\rightarrow$ swap



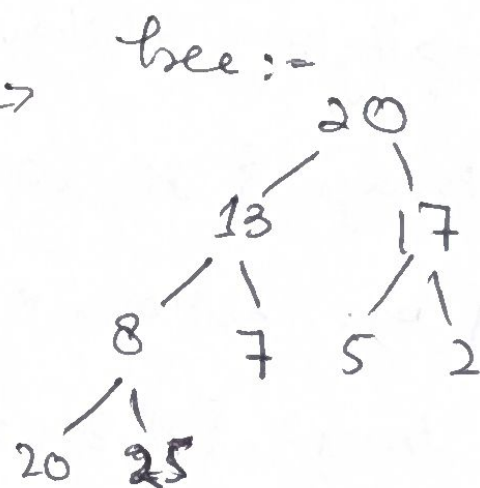
② —!

Swap $A[1] \leftrightarrow A[8]$



Heapify (8): Largest child = 17 $\rightarrow$ swap

$\rightarrow \{20, 13, 17, 8, 7, 5, 2, 20, 25\}$



③ — 1

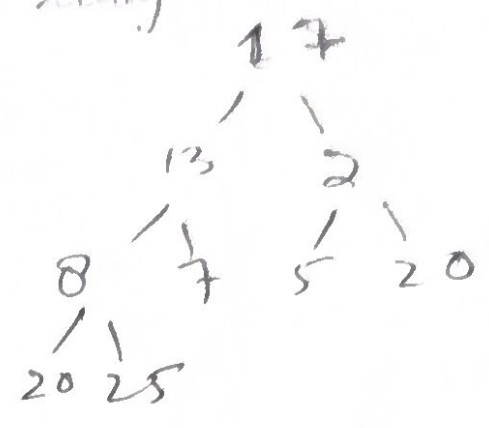
Swap $A[1] \leftrightarrow A[7]$

$\rightarrow \{2, 13, 17, 8, 7, 5, 20, 20, 25\}$

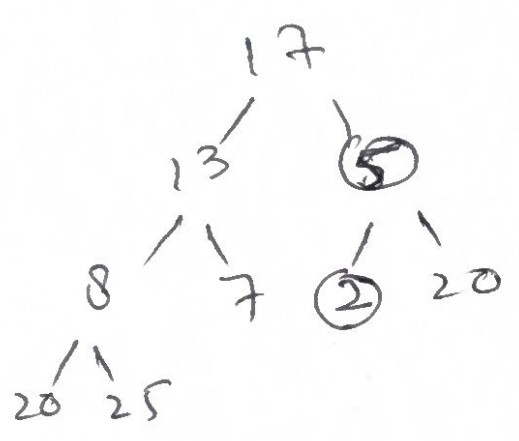
Heapify size 6:

Swap Index 3

$\rightarrow$ new array



Heapify(3): children 5 $\rightarrow$ largest 5 $\rightarrow$ swap.



④

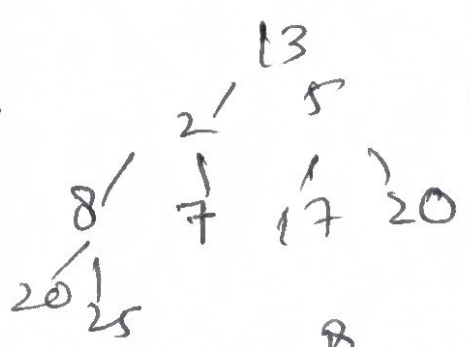
— 1

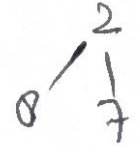
Swap $A[1] \leftrightarrow A[6]$

$\rightarrow \{2, 13, 5, 8, 7, 17, 20, 20, 25\}$

Heapify size 5.

largest 13 $\rightarrow$ swap



Heapify(2):  $\rightarrow$ 8 largest $\rightarrow$ 

$\rightarrow \{13, 8, 5, 2, 7, 17, 20, 20, 25\}$